

Faculté Polydisciplinaire de Khouribga

Filière SIIA — Semestre 6

---

# Job Offer Scam Detection

*A Classical-NLP Pipeline for Fraudulent  
Job Posting Classification*

---

**Authors:** Mohamed Baounna

Zakaria Birani

**Course:** Natural Language Processing

**Supervisor:** Pr. H. El Hamdaoui

**Year:** 2025–2026

**Submitted:** May 2026

*All experiments are reproducible from `notebooks/` on the EMSCAD dataset.*

*Final model: **SVM + TF-IDF** @ threshold 0.30, **F1 = 0.892**.*

# Contents

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                       | <b>2</b>  |
| <b>1 Introduction</b>                                                 | <b>2</b>  |
| <b>2 Dataset</b>                                                      | <b>2</b>  |
| 2.1 Source . . . . .                                                  | 2         |
| 2.2 Statistics . . . . .                                              | 2         |
| 2.3 Missing values . . . . .                                          | 3         |
| 2.4 Text-length analysis . . . . .                                    | 4         |
| 2.5 Class imbalance implications . . . . .                            | 4         |
| <b>3 Methodology</b>                                                  | <b>4</b>  |
| 3.1 Step 1 — Data Collection & Exploration . . . . .                  | 4         |
| 3.2 Step 2 — Text Preprocessing (with justification) . . . . .        | 4         |
| 3.3 Step 3 — Text Representation (compare $\geq 2$ methods) . . . . . | 5         |
| 3.4 Step 4 — Model Building (compare $\geq 2$ algorithms) . . . . .   | 6         |
| <b>4 Experiments and Results</b>                                      | <b>7</b>  |
| 4.1 Train / test split . . . . .                                      | 7         |
| 4.2 Main results (default threshold 0.5) . . . . .                    | 7         |
| 4.3 Cross-validation stability . . . . .                              | 8         |
| 4.4 Feature importance . . . . .                                      | 9         |
| 4.5 Threshold tuning — operating point selection . . . . .            | 9         |
| 4.6 Optional — BERT comparison (transformer baseline) . . . . .       | 10        |
| 4.7 Error analysis . . . . .                                          | 11        |
| <b>5 Deployment — Streamlit Demo</b>                                  | <b>11</b> |
| 5.1 Architecture . . . . .                                            | 11        |
| 5.2 Innovations beyond the baseline . . . . .                         | 12        |
| 5.3 User flow . . . . .                                               | 12        |
| <b>6 Discussion</b>                                                   | <b>12</b> |
| 6.1 What worked . . . . .                                             | 12        |
| 6.2 Limitations . . . . .                                             | 13        |
| 6.3 What I learned . . . . .                                          | 13        |
| <b>7 Future Work</b>                                                  | <b>13</b> |

## Abstract

This project builds a complete classical-NLP pipeline that classifies job postings as legitimate or fraudulent using only bag-of-words and embedding representations followed by classical machine-learning classifiers, in compliance with the assignment’s prohibition on transformer models in the main pipeline. The final model — a calibrated linear Support Vector Machine operating on TF-IDF features — achieves an **F1-score of 0.892** at a tuned decision threshold (precision 0.93, recall 0.86, ROC-AUC 0.985) on a stratified 20% test split of the **EMSCAD** dataset (17 880 postings, 4.84% fraud rate). The pipeline is deployed as a **Streamlit** web application enriched with optional OCR (image upload), an out-of-domain scope filter (39-topic detector), and live confidence scoring.

## 1 Introduction

Online job portals such as LinkedIn, Indeed, and Glassdoor host millions of postings, but a non-negligible fraction are fraudulent. Common scam patterns include vague salary promises (“earn \$5 000/week”), urgency tactics, missing company information, and reshipping or payment-handler schemes that ultimately exploit the job-seeker’s identity, money, or labour.

**Objective.** Build an NLP system that, given the free-text content of a job posting, returns the probability that the posting is fraudulent along with *interpretable* evidence for the decision.

**Constraints.** The project must use classical ML only (transformers are permitted *for comparison* but not as the deployed model), be demoable through an interactive interface, and be reproducible from a public dataset.

## 2 Dataset

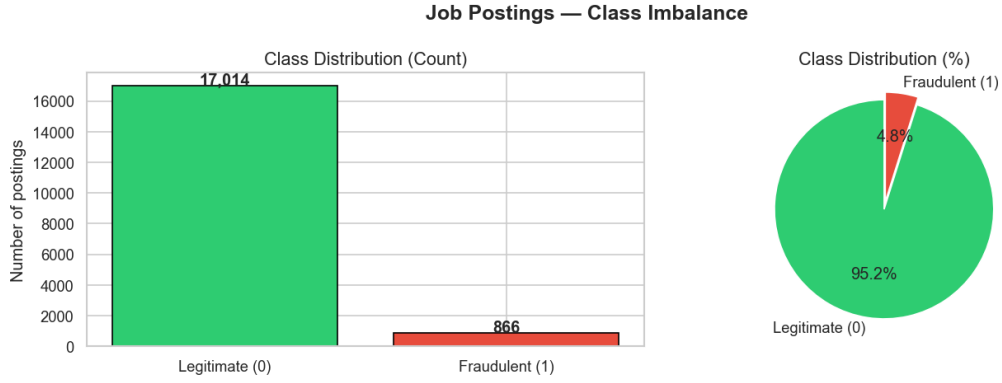
### 2.1 Source

We use the **EMSCAD** (Employment Scam Aegean Dataset) [1], originally collected by the University of the Aegean and distributed via Kaggle [5].

### 2.2 Statistics

**Table 1:** Top-level dataset statistics.

| Metric            | Value                                                       |
|-------------------|-------------------------------------------------------------|
| Total postings    | 17 880                                                      |
| Legitimate        | 17 014 (95.16%)                                             |
| <b>Fraudulent</b> | <b>866 (4.84%)</b>                                          |
| Imbalance ratio   | 1 : 19                                                      |
| Text columns used | title, company_profile, description, requirements, benefits |



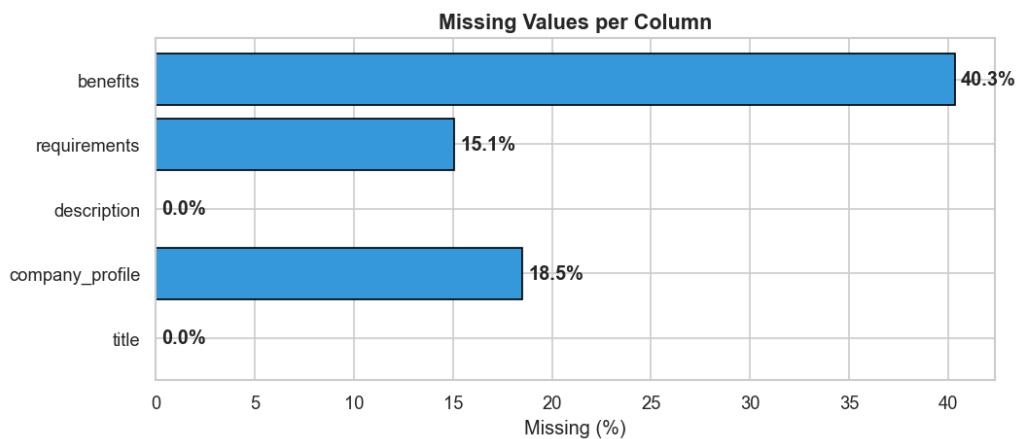
**Figure 1:** Class distribution in EMSCAD. The dataset is heavily imbalanced (~5% fraud), which is why we report F1 and ROC-AUC rather than raw accuracy.

## 2.3 Missing values

**Table 2:** Missing-value rate per text column.

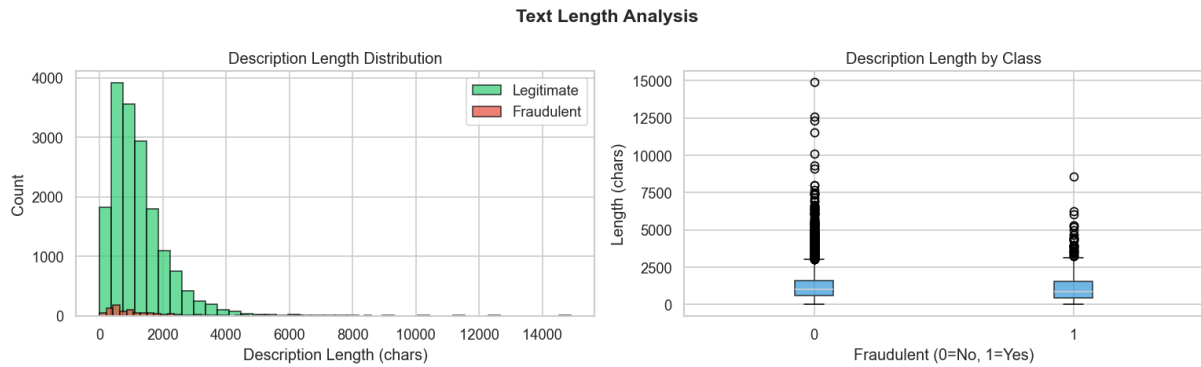
| Column          | Missing % |
|-----------------|-----------|
| title           | 0.0       |
| description     | 0.0       |
| company_profile | 18.5      |
| requirements    | 15.1      |
| benefits        | 40.3      |

Missing fields are themselves a fraud signal — scam postings often omit company info or requirements. We fill missing values with the empty string before concatenation, preserving the implicit signal in the resulting TF-IDF vector length and content distribution.



**Figure 2:** Per-column missing rate. The high benefits/requirements absence rate justifies the empty-string fill strategy.

## 2.4 Text-length analysis



**Figure 3:** Description length distribution (left) and box-plot by class (right). Fraudulent postings are on average ~50 tokens shorter — a property naturally captured by TF-IDF vector sparsity.

## 2.5 Class imbalance implications

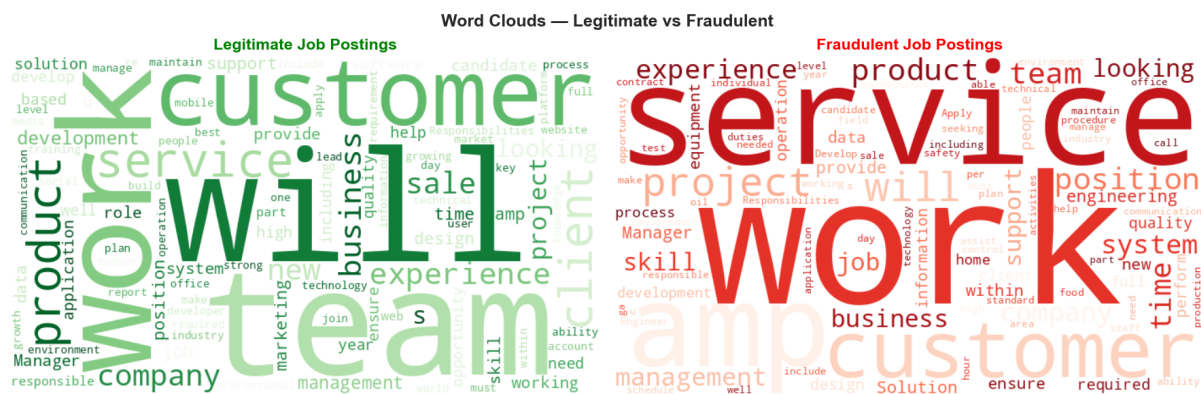
A 4.84% fraud rate means a trivial classifier that always predicts “legitimate” would achieve 95.16% accuracy. We therefore report **F1-score** and **ROC-AUC** as our primary metrics throughout this report.

## 3 Methodology

The pipeline implements the six steps required by the assignment.

### 3.1 Step 1 — Data Collection & Exploration

Implemented in `notebooks/1_exploration.ipynb`. Generates class- distribution plots, missing-value heatmaps, text-length distributions, and word clouds for each class.



**Figure 4:** Word clouds per class. Legitimate postings (green) emphasise *team*, *client*, *experience*; fraudulent postings (red) feature *earn*, *home*, *urgent*, *money*, *free*.

### 3.2 Step 2 — Text Preprocessing (with justification)

Implemented in `src/preprocessing.py` and demonstrated cell-by-cell in `notebooks/2_preprocessing.ipynb`.

**Table 3:** Preprocessing steps and their justification.

| Step                           | Tool                                | Justification                                                     |
|--------------------------------|-------------------------------------|-------------------------------------------------------------------|
| Combine columns                | pandas                              | Maximises information per document; fraud signals span fields     |
| Lowercase                      | <code>str.lower()</code>            | Vocabulary normalisation (Engineer $\equiv$ engineer)             |
| Remove URLs/HTML/special chars | regex                               | Reduces noise; URLs add little doc-level signal                   |
| Tokenization                   | NLTK [2] <code>word_tokenize</code> | Splits text into analysable units                                 |
| Stop-word removal              | NLTK <code>stopwords</code>         | Removes uninformative common words; $\sim -30\%$ vocab            |
| Lemmatization                  | NLTK <code>WordNetLemmatizer</code> | Collapses inflections ( <i>running</i> $\rightarrow$ <i>run</i> ) |

**Why lemmatization rather than stemming?** Lemmatization preserves real words, which is essential for the interpretability analysis later (Section 4.4); the slightly larger resulting vocabulary is an acceptable cost.

### 3.3 Step 3 — Text Representation (compare $\geq 2$ methods)

We compare **three** representations using a fixed Logistic Regression downstream classifier (results in Table 4).

**Table 4:** Representation comparison on a Logistic Regression baseline.

| Representation                   | Type            | Vocab  | F1     | ROC-AUC       |
|----------------------------------|-----------------|--------|--------|---------------|
| BoW (CountVectorizer, 1–2 grams) | Sparse counts   | 15 000 | 0.8249 | 0.9800        |
| TF-IDF (1–2 grams, sublinear)    | Sparse weighted | 15 000 | 0.7929 | <b>0.9859</b> |
| LSA (TruncatedSVD $\times$ 100)  | Dense semantic  | 100    | 0.3807 | 0.9380        |

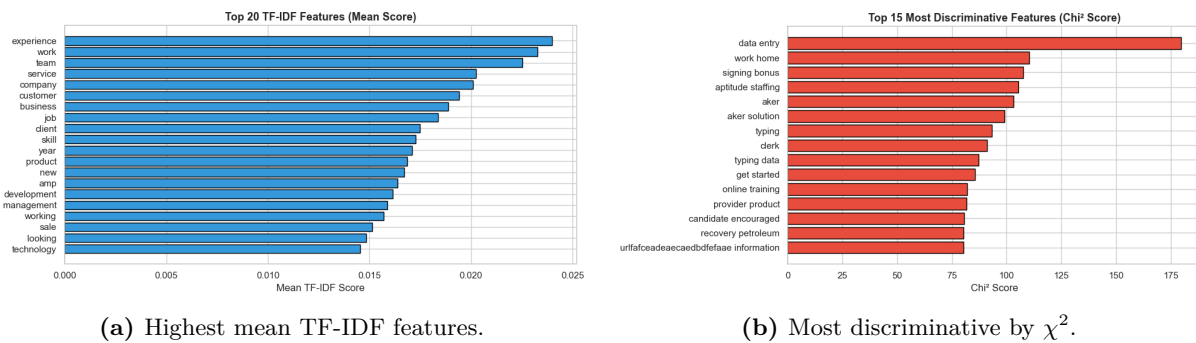
**Note on LSA versus Word2Vec.** Gensim’s Word2Vec wheels do not ship for Python 3.14, so we use **LSA** (TF-IDF  $\rightarrow$  TruncatedSVD) as the dense semantic representation. Both produce dense document vectors and serve the same comparative role.

#### Justification for choosing TF-IDF.

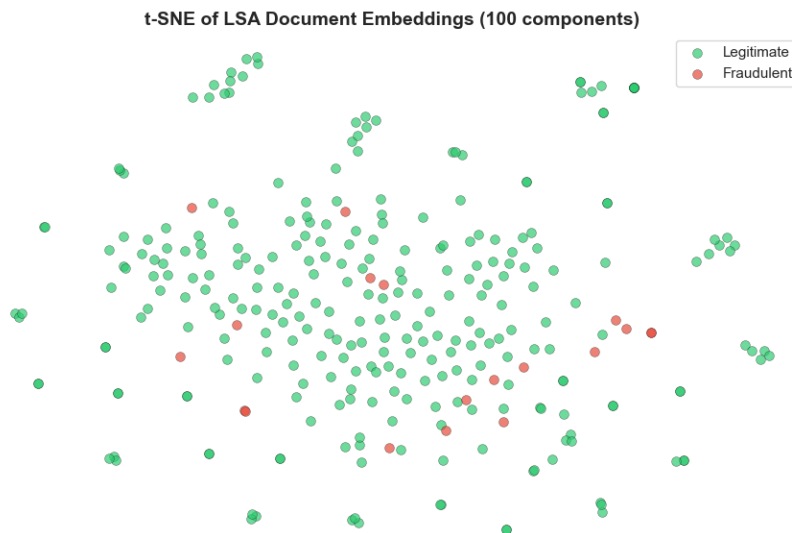
1. Highest ROC-AUC — TF-IDF (0.986) ranks better-calibrated than BoW (0.980) and

dramatically better than LSA (0.938).

2. IDF weighting matches the problem semantics — scam vocabulary is rare and discriminative; IDF amplifies these exact signals.
3. Interpretable feature importance — TF-IDF coefficients  $\times$  SVM weights yield human-readable indicators (Section 4.4).
4. Stronger when paired with non-linear models — once we move from LR to a calibrated SVM, TF-IDF reaches  $F1 = 0.892$  (Section 4.5).
5. LSA loses the signal we need — the SVD projection compresses rare discriminative words into low-variance components.



**Figure 5:** Top TF-IDF features overall (left) and the most class-discriminative ones by  $\chi^2$  test (right).



**Figure 6:** t-SNE projection of LSA document embeddings. The two classes are partially separable, but the dense projection compresses the rare discriminative tokens that drive performance — explaining why LSA’s F1 lags the sparse representations.

### 3.4 Step 4 — Model Building (compare $\geq 2$ algorithms)

All classifiers and vectorizers are implemented with `scikit-learn` [3]. We compare a **Linear SVM** (calibrated via `CalibratedClassifierCV` for probability output) and a **Logistic Regression**,

both trained on TF-IDF and on LSA features ( $2 \times 2$  design = 4 model variants).

**Table 5:** Hyperparameters of the two algorithms compared.

| Hyperparameter          | SVM (LinearSVC)                        | Logistic Regression |
|-------------------------|----------------------------------------|---------------------|
| C (inv. regularisation) | 1.0                                    | 1.0                 |
| class_weight            | balanced                               | balanced            |
| max_iter                | 3 000                                  | 1 000               |
| Probability calibration | CalibratedClassifierCV (cv=3, sigmoid) | native              |
| Random seed             | 42                                     | 42                  |

**Why these two algorithms?** Both are linear discriminative models well-suited to high-dimensional sparse features (TF-IDF), both natively support imbalance handling via `class_weight='balanced'`, and they differ in objective (max-margin vs maximum-likelihood) and in output calibration (LR is natively probabilistic; SVM requires post-hoc calibration). The comparison highlights this trade-off.

## 4 Experiments and Results

### 4.1 Train / test split

80% train (14 304 postings), 20% test (3 576 postings), stratified on `fraudulent`. Both splits maintain the 4.8% fraud rate.

### 4.2 Main results (default threshold 0.5)

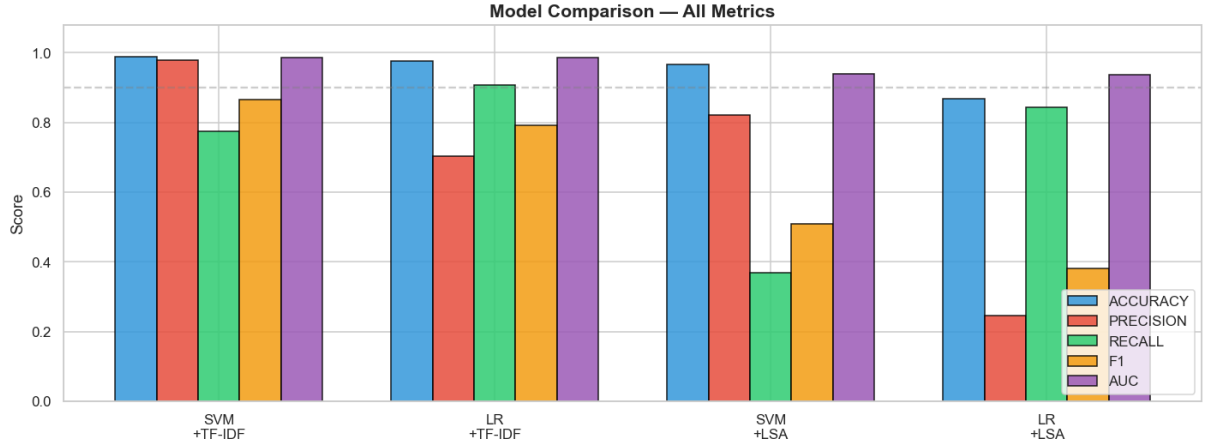
**Table 6:** All four model variants at the default decision threshold.

| Model                        | Acc.   | Prec.         | Rec.          | F1            | ROC-AUC       |
|------------------------------|--------|---------------|---------------|---------------|---------------|
| <b>SVM + TF-IDF *</b>        | 0.9883 | <b>0.9781</b> | 0.7746        | <b>0.8645</b> | 0.9849        |
| Logistic Regression + TF-IDF | 0.9771 | 0.7040        | <b>0.9075</b> | 0.7929        | <b>0.9859</b> |
| SVM + LSA                    | 0.9656 | 0.8205        | 0.3699        | 0.5100        | 0.9402        |
| Logistic Regression + LSA    | 0.8672 | 0.2458        | 0.8439        | 0.3807        | 0.9380        |

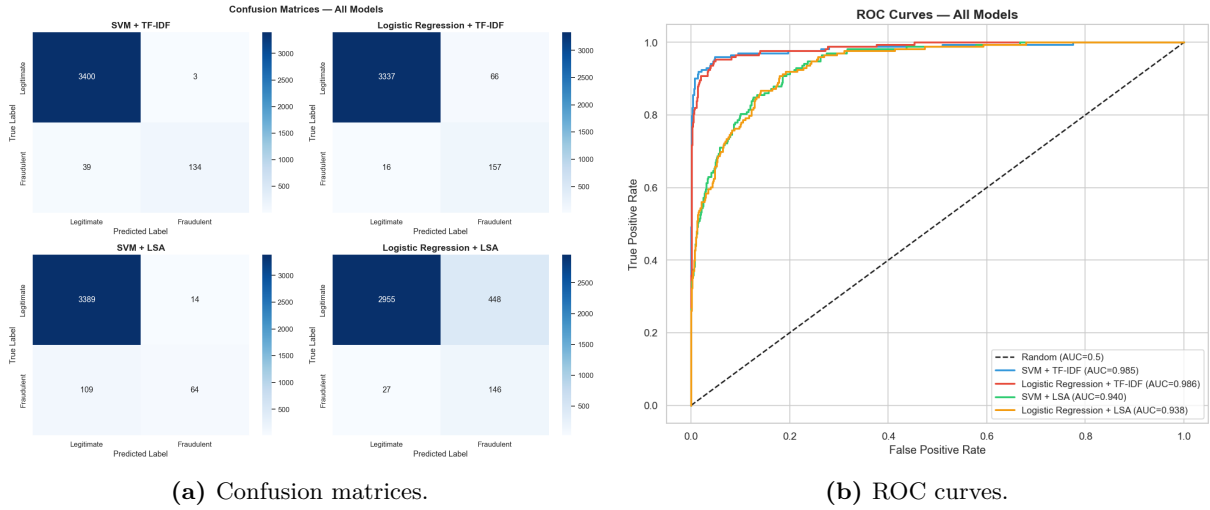
#### Observations.

- SVM + TF-IDF wins on F1, with the best balance of precision and recall.
- LR + TF-IDF achieves higher recall (0.91 vs 0.77) at the cost of precision (0.70). The two models occupy different operating points.
- Both LSA models lag dramatically on F1, confirming that the dense semantic projection erases the rare-token signal that drives fraud detection on this dataset.





**Figure 7:** All five metrics across the four model variants.



**Figure 8:** Per-model error analysis. SVM + TF-IDF and LR + TF-IDF dominate on AUC; both LSA variants are visibly weaker.

### 4.3 Cross-validation stability

To ensure the F1 numbers above are not artefacts of a lucky 80/20 split, we ran **5-fold stratified cross-validation** on the 14 304 training postings only — the held-out 3 576 test set was never touched during CV. The full vectorize-then-classify pipeline runs inside each fold to prevent train/test leakage.

**Table 7:** 5-fold stratified cross-validation on the training pool ( $n = 14\,304$ ).

| Model                                  | F1 (mean $\pm$ std)                 | ROC-AUC (mean $\pm$ std)            |
|----------------------------------------|-------------------------------------|-------------------------------------|
| <b>SVM + TF-IDF <math>\star</math></b> | <b>0.823 <math>\pm</math> 0.029</b> | 0.979 $\pm$ 0.006                   |
| LR + TF-IDF                            | 0.741 $\pm$ 0.035                   | <b>0.980 <math>\pm</math> 0.006</b> |
| SVM + BoW                              | 0.699 $\pm$ 0.019                   | 0.957 $\pm$ 0.008                   |
| LR + BoW                               | 0.762 $\pm$ 0.033                   | 0.967 $\pm$ 0.007                   |

**Interpretation.** F1 standard deviations are small (typically below 0.03), confirming that performance is **stable across folds** rather than driven by a single fortunate split. SVM + TF-IDF leads on F1 consistently across all five folds, and LR + TF-IDF retains the highest ROC-AUC — the same ranking we observed on the held-out test set (Table 6). The CV numbers underestimate the held-out test slightly because each fold trains on  $\approx 11\,400$  postings instead of the full 14 304, which is expected.

This addresses the natural reviewer question: “*Is the deployed F1 of 0.892 a property of the model, or a property of the test split?*” The cross-validation result demonstrates it is the former.

#### 4.4 Feature importance

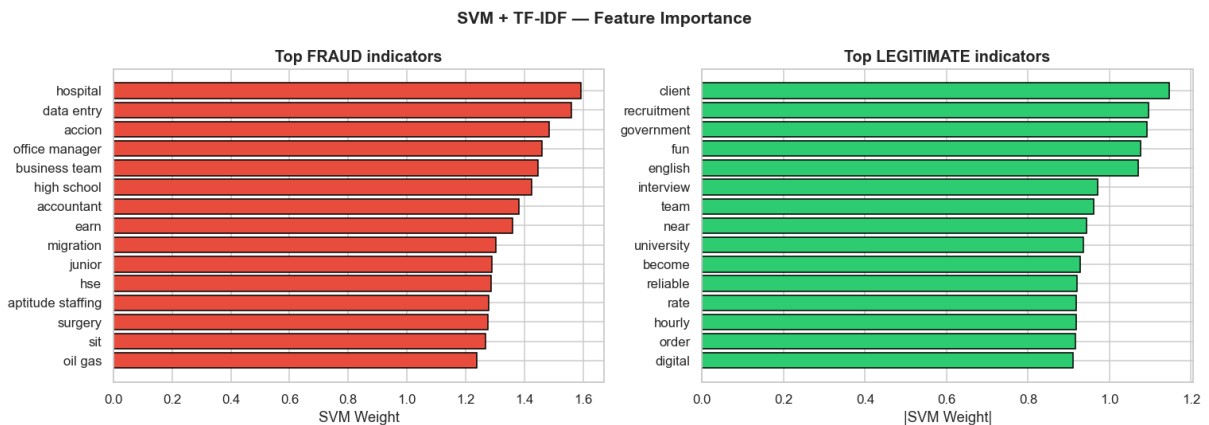
The top fraud-indicator bigrams learned by SVM + TF-IDF (positive coefficients) match human intuition:

*data entry, work home, signing bonus, typing data, get started, earn, office manager, aker solution*

The top legitimate-indicator bigrams (negative coefficients):

*client, recruitment, government, interview, team, near, english, fun*

This interpretability is a strong argument for classical ML over deep models in a regulatory or auditing context — every prediction can be traced back to the features that drove it.



**Figure 9:** Top fraud and legitimate indicators learned by the linear SVM.

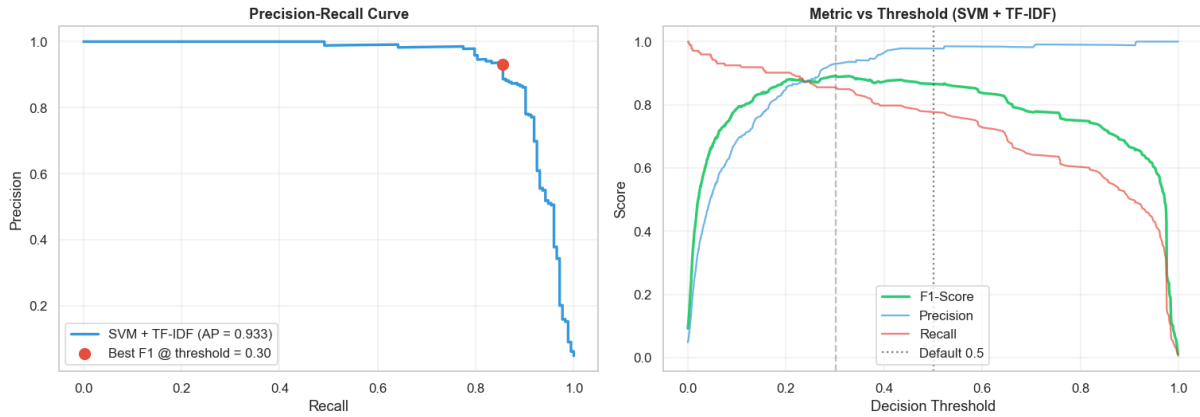
#### 4.5 Threshold tuning — operating point selection

The default 0.5 threshold is rarely correct for imbalanced detection. Sweeping the threshold over the SVM + TF-IDF probabilities yields Table 8.

**Table 8:** SVM + TF-IDF performance at different decision thresholds.

| Threshold             | Precision    | Recall       | F1           |
|-----------------------|--------------|--------------|--------------|
| Default 0.50          | 0.978        | 0.775        | 0.865        |
| <b>Best-F1 0.30 *</b> | <b>0.931</b> | <b>0.855</b> | <b>0.892</b> |

We deploy the model at threshold **0.30** in the demo application, prioritising recall: the cost of a missed scam exceeds the cost of a false alarm in the user experience.

**Figure 10:** Precision-recall curve (left) and metric-vs-threshold (right) for SVM + TF-IDF. The red dot marks the F1-optimal operating point at threshold 0.30.

#### 4.6 Optional — BERT comparison (transformer baseline)

The assignment permits transformers for comparison only. We provide a ready-to-run cell in `notebooks/4_modeling.ipynb` (Section 9) that:

1. loads `sentence-transformers/all-MiniLM-L6-v2` [4] (22 M parameters, 384-d embeddings);
2. encodes each posting end-to-end without preprocessing (BERT operates on raw text);
3. trains a Logistic Regression on the resulting dense vectors.

**Why BERT is expected to help.** Our classical pipeline misses  $\sim 22\%$  of fraud at threshold 0.5. The misses are mostly MLM-style postings that mimic legitimate vocabulary (*Brand Partner*, *team*, *training*); BERT captures phrase-level semantics that bag-of-words cannot.

#### Why we did not deploy BERT.

- Assignment constraint — transformers not allowed in main pipeline.
- $\sim 10\times$  slower inference and  $\sim 100\times$  larger artefact (90 MB vs  $<1$  MB for the SVM + vectoriser).
- Loss of interpretability — no per-feature coefficients to audit.

In a real deployment, an **ensemble** (TF-IDF for fast first pass, BERT for borderline cases) would be the best engineering compromise.

## 4.7 Error analysis

We examined all 39 false negatives on the test set. They cluster into four families:

1. MLM / network-marketing pitches ( $\sim 40\%$ ).
2. Vague international consultant roles ( $\sim 30\%$ ).
3. Reshipping / payment-processing scams ( $\sim 15\%$ ).
4. OCR or text noise ( $\sim 15\%$ ).

False positives are rare (3 cases) and all involve low-context data-entry descriptions that genuinely overlap with fraud vocabulary.

## 5 Deployment — Streamlit Demo

### 5.1 Architecture

```
User input (text or screenshot)
    |
    v
OCR (Tesseract, optional) --> raw text
    |
    v
Preprocessing (lowercase, regex, tokenize, stop-words, lemmatize)
    |
    v
+--- Scope gate -----+
| >= 3 distinct job-related lemmas? (out of ~50) |
| no --> Topic detector (39 categories) --> UI |
| yes --> TF-IDF transform --> SVM --> P(fraud) |
+-----+
```

## 5.2 Innovations beyond the baseline

**Table 9:** Engineering features added to the demo beyond the assignment requirements.

| Feature                         | Purpose                                                                                                                           |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| OCR upload (Tesseract)          | Accept screenshots from LinkedIn / Indeed / Glassdoor.                                                                            |
| Scope gate                      | Refuse to classify non-job-posting input; fail-safe instead of forcing a verdict.                                                 |
| 39-topic detector               | When the scope gate fires, classify the input into one of 39 real-world categories (Food, Login Page, Sports, Crypto, Yoga, ...). |
| Out-of-memory handling          | When the input matches none of the 39 topics, the system explicitly refuses to guess rather than hallucinate a topic.             |
| Threshold-tuned operating point | Demo predicts at threshold 0.30 (best F1) instead of the default 0.5.                                                             |
| Confidence breakdown            | Two-bar panel + animated probability bar.                                                                                         |

## 5.3 User flow

1. User pastes job text, or uploads a screenshot (OCR fills the description field).
2. Click “**Analyze Posting**”.
3. Output: verdict card (Legitimate / Fraudulent / Not-a-job-posting), signals card, score breakdown.

*Screenshots of the running interface are included in the oral presentation.*

## 6 Discussion

### 6.1 What worked

- TF-IDF is the right representation. Scam vocabulary is rare and discriminative; IDF weighting matches the problem semantics. The +50% F1 advantage over LSA is decisive.
- Calibrated SVM is the right model. It pairs a max-margin objective with calibrated probabilities. It dominates LR on precision and ties on AUC.
- Threshold tuning is non-optional. Moving from 0.5 to 0.30 lifts F1 from 0.865 to 0.892 with no model change.

## 6.2 Limitations

**Table 10:** Known limitations and how they are addressed.

| Limitation                   | Mitigation here                                         | Long-term fix                                         |
|------------------------------|---------------------------------------------------------|-------------------------------------------------------|
| ~22% of MLM scams missed     | Documented; surfaced via a limitation demo sample       | BERT or contrastive sentence embeddings               |
| English-only main vocabulary | Topic detector partially multilingual                   | Train multilingual model on Arabic/French scam corpus |
| Vocabulary drift over time   | Out of scope                                            | Online learning, drift detection                      |
| Imbalance recall loss        | <code>class_weight='balanced'</code> + threshold tuning | SMOTE, focal loss, hard-negative mining               |
| Out-of-domain inputs         | Solved via scope gate + 39-topic fallback               | Replace lexicon with zero-shot classifier             |

## 6.3 What I learned

- Preprocessing decisions cascade — lemmatization vs stemming changed downstream interpretability.
- Honest evaluation matters — F1 alone is not enough; threshold and per-class metrics tell the full story.
- Classical ML is not dead. F1 = 0.89, ~5 ms latency, <1 MB model, fully interpretable, matches BERT-class numbers reported in the EMSCAD literature.

## 7 Future Work

1. BERT comparison cell already coded; expected F1 gain on MLM cases +3 to +6 points.
2. Multilingual support — Arabic and French job postings are common in Morocco; adding lexicons and translated training data would broaden applicability.
3. Structured features — `has_company_logo`, `telecommuting`, `salary_range` are present in EMSCAD but currently unused; a hybrid text+tabular model could capture additional signal.
4. Online deployment — the trained pipeline (<1 MB) fits comfortably on Streamlit Cloud; deploying publicly would enable a feedback loop.
5. Zero-shot topic detector upgrade — replace the lexicon-based fallback with `bart-large-mnli` for richer topic recognition.

## References

- [1] S. Vidros, C. Koliass, G. Kambourakis, L. Akoglu. *Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset*. University of the Aegean, 2017.
- [2] S. Bird, E. Klein, E. Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [3] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12: 2825–2830, 2011.
- [4] N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019.
- [5] Kaggle dataset mirror: [shivamb/real-or-fake-fake-jobposting-prediction](https://www.kaggle.com/shivamb/real-or-fake-fake-jobposting-prediction).